Random Video Chat Platform – Project Requirements Document

Version: 1.1 (Draft) **Date:** October 17, 2026 **Stack:** Go microservices, gRPC, event bus, Redis, PostgreSQL, Kubernetes, WebRTC

1. Introduction

1.1 Purpose

This document defines the requirements for a web-based random video and text chat platform in the spirit of Omegle, built as a set of independently deployable microservices. Authenticated users are paired with a random eligible partner, talk over live audio and video (or text only), and can skip to a new partner at any time. The goal is Omegle-style feature parity with a faster, safer and horizontally scalable implementation.

1.2 Scope

In scope (MVP): account registration and login, random one-to-one matchmaking, video chat and text-only chat modes, interest-based matching with a random fallback, WebRTC audio and video, in-session text chat with typing indicators, skip and disconnect, reconnection handling, reporting and blocking, rate limiting and bot protection, moderation tooling, and the microservice platform (API gateway, event bus, observability and autoscaling).

Out of scope for MVP: group rooms, native mobile apps, session recording, monetisation and premium features, and any unmoderated or adult-oriented section.

1.3 Definitions

Term	Meaning
WebRTC	Browser technology for real-time audio, video and data transfer between peers
Signaling	Exchange of connection setup messages (SDP offers and answers, ICE candidates) coordinated by the backend
STUN / TURN	Servers that help peers discover public addresses (STUN) or relay media when a direct connection fails (TURN)

Term	Meaning
Matchmaking	Server-side logic that pairs two waiting users
Session	A single conversation between two matched users
Microservice	A small, independently deployable service that owns one capability and its own data
API Gateway	Single entry point for REST calls: authentication, routing and rate limiting
Realtime Gateway	Entry point that holds WebSocket connections and routes real-time messages
Event bus	Message backbone that carries facts such as SessionEnded between services
gRPC	Binary request/response protocol used between services
SLO	Service level objective: a measurable performance or availability target

1.4 Key Principle

WebRTC is not the matchmaking system. WebRTC only carries real-time media. The Matchmaking Service decides who is connected to whom, and the Signaling Service coordinates how their browsers connect.

2. Product Overview

2.1 How the Platform Works

1. **Authentication:** the platform verifies the user and their session.
2. **Random matchmaking:** a waiting user is paired with another eligible user.
3. **Signaling:** the backend coordinates the browsers' WebRTC connection setup.
4. **Live conversation:** browsers exchange audio and video peer to peer (or via TURN relay).
5. **Skip or disconnect:** the session ends and the user can search for a new partner.

2.2 Design Choices vs. Standard Approach

Component	Classification
Random one-to-one matching	Core concept shared with Omegle-style products
Text chat and video chat modes	Core Omegle experience
Interest tags with random fallback	Omegle feature
Server-mediated signaling	Standard WebRTC architecture
Browser-based audio and video	Standard WebRTC architecture
Skip and find another person	Core user experience
Microservices behind an API gateway and event bus	Architecture choice
Go services	Technology choice
Redis atomic matchmaking	Implementation choice
PostgreSQL per service for accounts and moderation	Implementation choice
Mandatory authentication	Product choice (the classic experience was anonymous); optional guest mode is behind a feature flag

2.3 Improvements over a Basic Clone

- **Race-free matching:** a user can never be assigned to two conversations at once.
- **Fast skip:** the old match is invalidated before a new match can begin.
- **Reliable reconnection:** temporary network failures never leave users stuck in stale sessions.
- **Abuse prevention:** reporting, blocking, rate limits and moderation.
- **Scalability:** independent microservices scale horizontally, and Redis coordinates matching across instances.
- **Privacy:** minimal data collection and no default video recording.

3. Stakeholders and Users

Role	Description	Key needs
End user	Verified adult who wants to meet new people over video	Quick matching, smooth video, easy skip, feeling of safety
Moderator	Reviews reports and takes action	Report queue, context, ban tools
Administrator	Operates the platform	Monitoring, configuration, user management
Developer / DevOps	Builds and maintains the system	Clear architecture, observability, deployment automation

4. Functional Requirements

Priority: **M** = Must (MVP), **S** = Should, **C** = Could (later phase).

4.1 Authentication and Accounts

ID	Requirement	Priority
FR-AUTH-01	Users can register with email and password, with email verification	M
FR-AUTH-02	Users can log in and log out; sessions use short-lived access tokens plus refresh tokens	M
FR-AUTH-03	Passwords are hashed with a modern adaptive algorithm (Argon2id or bcrypt)	M
FR-AUTH-04	Users must confirm they are 18 or older at registration and accept Terms and Community Guidelines	M
FR-AUTH-05	Password reset via emailed one-time link	M
FR-AUTH-06	Social login (e.g. Google)	C
FR-AUTH-07	Users can delete their account and associated personal data	M

4.2 Matchmaking

ID	Requirement	Priority
FR-MATCH-01	An authenticated user can press Start to join the waiting pool	M
FR-MATCH-02	The Matchmaking Service pairs two waiting users atomically using Redis (Lua script or equivalent atomic operation)	M
FR-MATCH-03	A user can belong to at most one waiting entry and at most one active session at any time	M
FR-MATCH-04	Users are never matched with themselves, with users they have blocked, or with users who blocked them	M
FR-MATCH-05	Recently matched partners are excluded for a configurable cool-down period to avoid immediate rematches	S
FR-MATCH-06	A user waiting too long sees a status message and may cancel the search	M
FR-MATCH-07	Banned or suspended users cannot join the pool and are removed from it immediately when suspended	M
FR-MATCH-08	Users can add up to 5 interest tags; the matcher prefers a partner who shares at least one tag and falls back to pure random matching after a short timeout (default 5 seconds)	M
FR-MATCH-09	Matched users are shown the interests they have in common	S
FR-MATCH-10	Chat mode (text or video) is part of the match key, so users only match within the same mode	M
FR-MATCH-11	Language preference filter; the default is the browser language	S
FR-MATCH-12	Waiting queues are sharded by mode, language and interest bucket so each queue stays small and fast	M
FR-MATCH-13	Region preference filter	C

4.3 Signaling and Live Conversation

ID	Requirement	Priority
FR-RTC-01	The backend exposes a WebSocket signaling channel authenticated with the user's token	M
FR-RTC-02	The backend relays SDP offers/answers and ICE candidates only between the two users of the same active session	M
FR-RTC-03	Browsers establish audio and video using WebRTC, with STUN and TURN servers configured	M
FR-RTC-04	Users can mute/unmute the microphone and enable/disable the camera during a session	M
FR-RTC-05	Users can exchange text messages during a session over the signaling channel or a WebRTC data channel	M
FR-RTC-06	Media quality adapts to network conditions; the UI shows connection state (connecting, connected, poor connection, disconnected)	M
FR-RTC-07	Camera and microphone permissions are requested with clear explanations, and errors are handled gracefully	M

4.4 Skip, Disconnect and Reconnection

ID	Requirement	Priority
FR-SESS-01	A Skip button ends the current session and immediately starts a new search	M
FR-SESS-02	On skip, the old session is invalidated server-side before the user re-enters the pool; late signaling messages for the old session are rejected	M
FR-SESS-03	The partner is notified immediately when the other user skips or disconnects, and is offered to find a new match	M
FR-SESS-04	If a client loses its connection, the server holds the session for a short grace period (default 10–15 seconds) to allow reconnect	M

ID	Requirement	Priority
FR-SESS-05	If the grace period expires, the session is ended, the partner is notified, and all session state is cleaned up	M
FR-SESS-06	A heartbeat mechanism detects dead connections so stale sessions and stale waiting entries are removed automatically	M
FR-SESS-07	If direct peer connection fails, the client attempts ICE restart or TURN relay before declaring failure	S

4.5 Safety, Reporting and Moderation

ID	Requirement	Priority
FR-SAFE-01	Users can report their current or most recent partner with a reason category and optional comment	M
FR-SAFE-02	Users can block a partner so they are never matched again	M
FR-SAFE-03	Rate limits apply to registration, login, matching requests, skip frequency and reports	M
FR-SAFE-04	Moderators have a dashboard with a report queue, user history and actions (warn, suspend, ban)	M
FR-SAFE-05	Repeat offenders are escalated automatically (for example temporary suspension after a threshold of validated reports)	S
FR-SAFE-06	When a report is submitted, a short rolling evidence snapshot (such as recent text messages and session metadata) may be preserved for review, disclosed in the privacy policy	S
FR-SAFE-07	Automated screening for nudity or other prohibited content on video frames, performed in a privacy-conscious way	C
FR-SAFE-08	A documented process for handling and escalating illegal content, including child safety, to the relevant authorities as required by law	M

ID	Requirement	Priority
FR-SAFE-09	Age assurance beyond self-declaration (for example ID or third-party age verification) evaluated before launch based on target markets and regulation	S
FR-SAFE-10	Users can appeal moderation decisions	S

4.6 Administration

ID	Requirement	Priority
FR-ADM-01	Administrators can view live metrics (online users, queue size, active sessions, average wait time)	S
FR-ADM-02	Administrators can manage moderator accounts and roles	M
FR-ADM-03	Configuration values (cool-downs, rate limits, grace periods) can be adjusted without redeploying	S

4.7 Omegle-Style Experience and Chat Modes

The platform reproduces the familiar Omegle experience feature by feature; the table lists each one and how it is delivered.

ID	Feature	Requirement	Priority
FR-OME-01	Text chat mode	Text-only chat with a random partner, with no camera or microphone needed	M
FR-OME-02	Video chat mode	Video with text chat beside it, as in classic Omegle	M
FR-OME-03	Interests	Optional interest tags before starting; shared tags are highlighted on match (see FR-MATCH-08 and FR-MATCH-09)	M
FR-OME-04	Anonymous partner	Partners appear only as "Stranger" and "You"; no username, email or profile data is ever shown	M

ID	Feature	Requirement	Priority
FR-OME-05	System messages	Familiar status lines: looking for someone to chat with, you are now chatting with a random stranger, stranger has disconnected	M
FR-OME-06	Typing indicator	"Stranger is typing…" shown in real time and rate-limited	M
FR-OME-07	Two-step disconnect	Skip button and Esc key; a second press confirms, to prevent accidental skips (configurable)	S
FR-OME-08	New chat	One click starts a new search; the ended chat stays readable until the new search starts	M
FR-OME-09	Online counter	Approximate count of users currently online	S
FR-OME-10	Chat transcript	Users can save their own chat text locally; the server does not store message content	S
FR-OME-11	Video controls	Mute, camera off, full screen, mirrored self-view and swap of local and remote picture	S
FR-OME-12	Safety banner	Persistent notice not to share personal information, with a one-click report button	M
FR-OME-13	Bot protection	CAPTCHA or proof-of-work challenge on suspicious behaviour; links from new accounts are blocked	M
FR-OME-14	Guest mode	Optional anonymous entry without an account, text-only, with CAPTCHA and stricter limits; feature flag, off by default	C
FR-OME-15	College mode	Match only users with a verified student email domain	C
FR-OME-16	Question mode	One user asks a question and two others discuss it while the asker watches	C

FR-OME-01 raises FR-RTC-05 (text chat during a session) to a Must.

Deliberately not replicated: the unmoderated section and any adult-oriented mode of the original service. Every session follows the same moderation, reporting and 18+ rules. The original service closed in 2023 amid litigation over misuse, which is why safety is a core feature here.

5. Non-Functional Requirements

5.1 Performance

- **NFR-PERF-01:** Matching latency under 500 ms at the 95th percentile (under 1 second at the 99th) when an eligible partner is waiting.
- **NFR-PERF-02:** Signaling message relay under 100 ms at the 95th percentile and under 250 ms at the 99th, within a region.
- **NFR-PERF-03:** Time from match to first video frame under 3 seconds for 90 percent of direct connections and under 5 seconds for relayed ones.
- **NFR-PERF-04:** Skip processed server-side in under 300 ms at the 95th percentile; the interface reacts instantly and shows the new search without waiting for the server.
- **NFR-PERF-05:** API Gateway adds under 10 ms at the 95th percentile; other REST calls complete in under 200 ms at the 95th percentile.
- **NFR-PERF-06:** Chat page loads in under 2 seconds on a 4G connection, with a script bundle under 250 KB compressed, code splitting and lazy loading of moderation and settings screens.
- **NFR-PERF-07:** Each Realtime Gateway instance sustains at least 20,000 concurrent WebSocket connections (target, confirmed by load test).
- **NFR-PERF-08:** Video targets 720p at 30 fps and adapts down to 240p on weak networks, with a default cap of 1.5 Mbps per direction.
- **NFR-PERF-09:** The match-and-signal hot path makes no PostgreSQL call and never waits on moderation, analytics or email.

5.2 Scalability

- **NFR-SCALE-01:** Every service is stateless; shared state lives in Redis, PostgreSQL and the event bus. Services scale independently using autoscaling on CPU, active connections and queue depth.
- **NFR-SCALE-02:** Realtime Gateway instances can be added at any time; a message for a user on another instance is routed through Redis or the event bus by session ID.
- **NFR-SCALE-03:** Initial target of 5,000 concurrent users, with an architecture that reaches 100,000 or more by adding nodes, without redesign.
- **NFR-SCALE-04:** Waiting queues are sharded by mode, language and interest bucket across a Redis Cluster, so there is no global lock or single hot key.
- **NFR-SCALE-05:** TURN capacity is sized for the share of sessions that cannot connect directly (commonly 10 to 20 percent), since relayed media is bandwidth-heavy.
- **NFR-SCALE-06:** The deployment is multi-region ready: users are routed to the nearest region, and matching runs per region in the MVP.

5.3 Reliability and Availability

- **NFR-REL-01:** Target availability of 99.5 percent for the MVP.
- **NFR-REL-02:** Graceful degradation: if Redis is unavailable, new matching stops with a clear message and no corrupted sessions are created.
- **NFR-REL-03:** All session-ending paths (skip, disconnect, timeout, crash) result in consistent cleanup with no orphaned state.

5.4 Security

- **NFR-SEC-01:** All traffic uses TLS; WebSockets use WSS; WebRTC media is encrypted by default (DTLS-SRTP).
- **NFR-SEC-02:** Time-limited TURN credentials are issued per session; static TURN credentials are never exposed to clients.
- **NFR-SEC-03:** Input validation, CORS and CSRF protection, and secure HTTP headers on all endpoints.
- **NFR-SEC-04:** Peer IP exposure is considered: users are informed that direct WebRTC connections can reveal IP addresses, and a TURN-relay-only option is evaluated to protect privacy.
- **NFR-SEC-05:** Secrets are stored in a secrets manager, never in source control.
- **NFR-SEC-06:** Regular dependency scanning and a documented vulnerability response process.
- **NFR-SEC-07:** Service-to-service traffic uses mutual TLS, and network policies allow only the calls listed in the service catalogue.
- **NFR-SEC-08:** The gateway validates the user token once; internal services receive a signed identity context and check authorization for their own data.

5.5 Privacy and Compliance

- **NFR-PRIV-01:** No audio or video recording by default.
- **NFR-PRIV-02:** Data minimisation: collect only what is needed for authentication, safety and operations.
- **NFR-PRIV-03:** Defined retention periods for logs, reports and evidence; automatic deletion afterward.
- **NFR-PRIV-04:** Privacy policy, Terms of Service and Community Guidelines published and accepted before first use.
- **NFR-PRIV-05:** Support for data access and deletion requests in line with applicable laws (such as GDPR and India's DPDP Act) in the markets served.

5.6 Usability and Compatibility

- **NFR-USE-01:** Works on current versions of Chrome, Edge, Firefox and Safari, on desktop and mobile browsers.
- **NFR-USE-02:** Responsive layout; a single prominent action for Start and Skip.
- **NFR-USE-03:** Accessible UI following WCAG 2.1 AA where practical (keyboard navigation, contrast, captions for text chat).

5.7 Observability and Maintainability

- **NFR-OBS-01:** Structured logging, metrics (Prometheus-compatible) and distributed tracing.
- **NFR-OBS-02:** Alerts on queue growth, match failure rate, signaling errors and TURN saturation.
- **NFR-MNT-01:** Automated unit, integration and load tests with CI/CD deployment pipeline.

5.8 Performance Engineering

The performance targets above are met by keeping the real-time path short and by isolating everything slow from it.

Area	Technique	Benefit
Matchmaking	One atomic Redis script per join, sharded queues, block lists cached in Redis	One round trip per pairing, no cross-queue contention
Signaling	Small JSON messages, relay by session ID, membership checked against a cached session record	Low relay latency without database reads
Hot path	Token verified at the gateway; no PostgreSQL, moderation or analytics call between Start and first frame	Latency independent of slow services
Media	Peer-to-peer first, regional coturn nodes as fallback, ICE trickle, TURN over TLS on port 443 as last resort	Fast connects and fewer failures behind strict firewalls
Client	Camera permission and signaling socket prepared on page load, credentials prefetched, optimistic skip	Faster perceived match

Area	Technique	Benefit
Network	HTTP/2 or HTTP/3 at the edge, CDN for static files, Brotli compression	Fast first load
Go runtime	Bounded per-connection buffers, write deadlines, backpressure, object reuse on hot paths, profiling in staging	Stable memory and latency under load
Database	Connection pooling, indexed lookups, read replicas for moderator screens, logs written asynchronously from events	Database never blocks chat
Load control	Autoscaling before peak hours; when overloaded, new searches get a busy message while active sessions are protected	Graceful behaviour at saturation

Capacity example: 5,000 concurrent users form 2,500 sessions. If 20 percent are relayed at 1.5 Mbps per direction, each relayed session sends 3 Mbps through TURN, so egress is about 500 x 3 = 1.5 Gbps.

6. Microservice Architecture

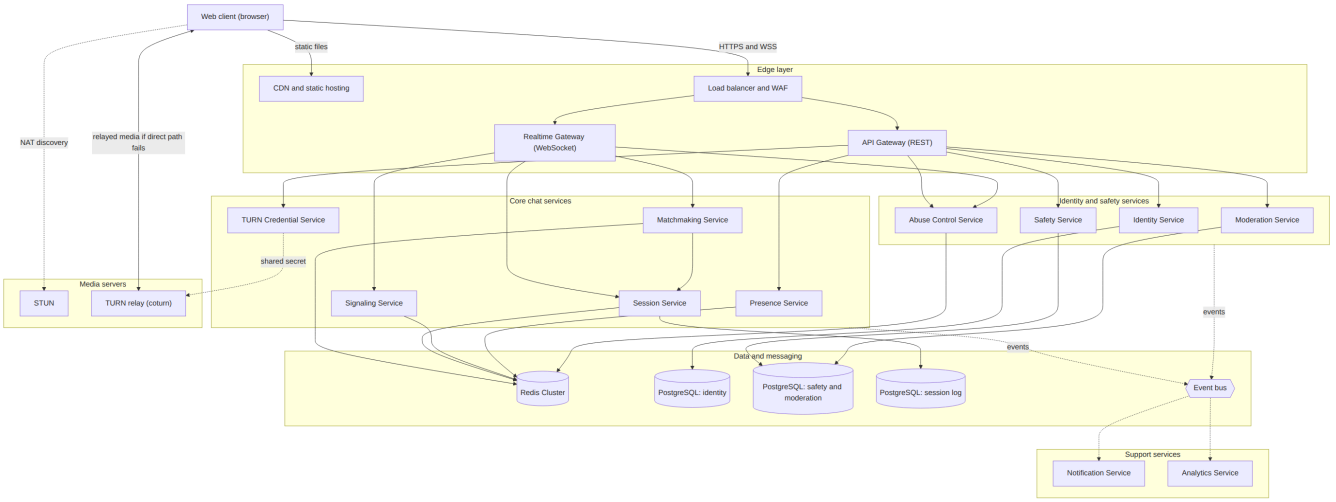
6.1 Principles

The platform is split into small services that each own one capability and one data store, talk through gRPC for requests and an event bus for facts, and sit behind two gateways.

- **Single responsibility:** one service per domain (identity, matchmaking, sessions, signaling, safety and so on).
- **Database per service:** no shared tables; other services use an API or events and refer to users by ID only.
- **Stateless services:** state lives in Redis, PostgreSQL and the event bus, so any instance can serve any request.
- **Synchronous for commands, asynchronous for facts:** gRPC for "create this session"; events such as SessionEnded for "this happened".
- **Hot path isolation:** matching and signaling never wait on moderation, analytics or email.
- **Failure isolation:** timeouts, retries with backoff, circuit breakers and bulkheads, so a failing support service cannot stop chat.

- **Independent delivery:** each service has its own container image, pipeline and versioned contract.
- **Observable and secure by default:** tracing with correlation IDs, metrics on every service, mutual TLS between services.

6.2 System Architecture



Clients reach the system through two entry points: the API Gateway for request and response calls and the Realtime Gateway for WebSocket traffic. Every other service stays on the private network, and only the Matchmaking, Session and Signaling services sit on the live chat path.

6.3 Microservice Catalogue

Thirteen services, each with one owner of its data. The MVP ships ten of them; Abuse Control, Presence and Analytics follow in Phase 2 (section 11).

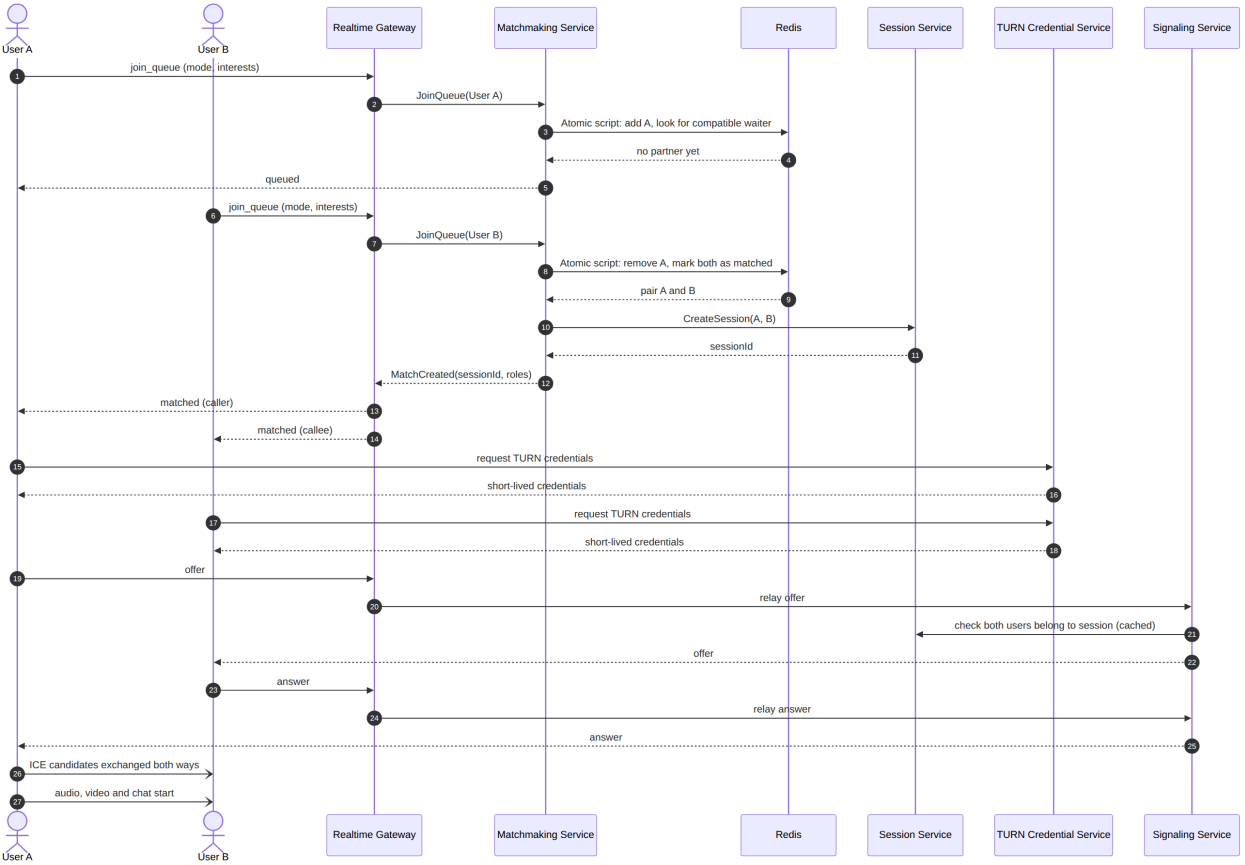
Service	Responsibility	Data store	Events published	Events consumed
API Gateway	TLS, token validation, routing, request rate limits, request IDs	None	None	None

Service	Responsibility	Data store	Events published	Events consumed
Realtime Gateway	Holds WebSocket connections, heartbeats and backpressure; routes client messages to services and pushes server events to clients	Redis (connection routing)	UserDisconnected, UserReconnected	MatchCreated, SessionEnded
Identity Service	Registration, login, tokens, email verification, password reset, 18+ confirmation, account deletion	PostgreSQL (identity)	UserRegistered, UserDeleted	UserSuspended, UserBanned
Matchmaking Service	Queues, atomic pairing, filters for mode, language, interests, blocks and cool-down	Redis (queues, cool-down)	MatchCreated	UserSuspended, BlockAdded
Session Service	Session state machine, one-session-per-user rule, reconnect grace period, cleanup, session metadata log	Redis (active), PostgreSQL (session log)	SessionStarted, SessionEnded	UserDisconnected, UserReconnected

Service	Responsibility	Data store	Events published	Events consumed
Signaling Service	Relays offers, answers, ICE candidates, chat messages and typing events between the two users of one session	Redis (session cache)	None	SessionEnded
TURN Credential Service	Issues short-lived STUN and TURN credentials per session	None	None	None
Safety Service	Reports, blocks, short evidence snapshots on report	PostgreSQL (safety)	ReportFiled, BlockAdded	SessionEnded
Moderation Service	Report queue, decisions, suspensions, bans, appeals, moderator and admin tools	PostgreSQL (safety)	UserSuspended, UserBanned	ReportFiled, RiskFlagged
Notification Service	Email for verification, password reset and moderation notices	None (provider queue)	None	UserRegistered, UserSuspended, UserBanned
Abuse Control Service	Rate limits, bot detection, CAPTCHA challenges, risk scoring	Redis	RiskFlagged	SessionEnded, ReportFiled

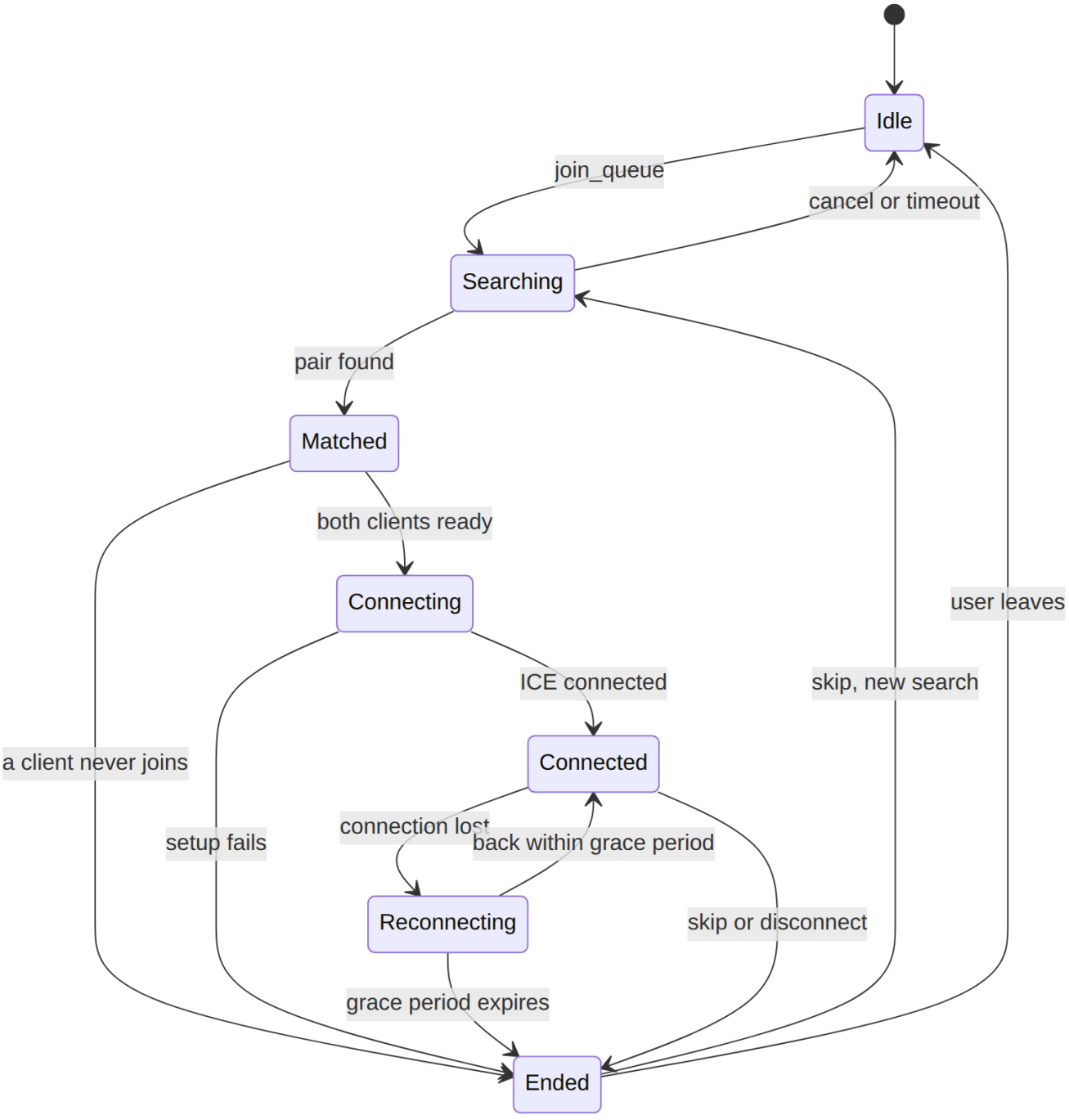
Service	Responsibility	Data store	Events published	Events consumed
Presence Service	Online user count and heartbeat aggregation	Redis	None	UserDisconnected, UserReconnected
Analytics Service	Turns events into metrics and dashboards; stores no message or media content	Time-series store	None	All events

6.4 Matchmaking Flow

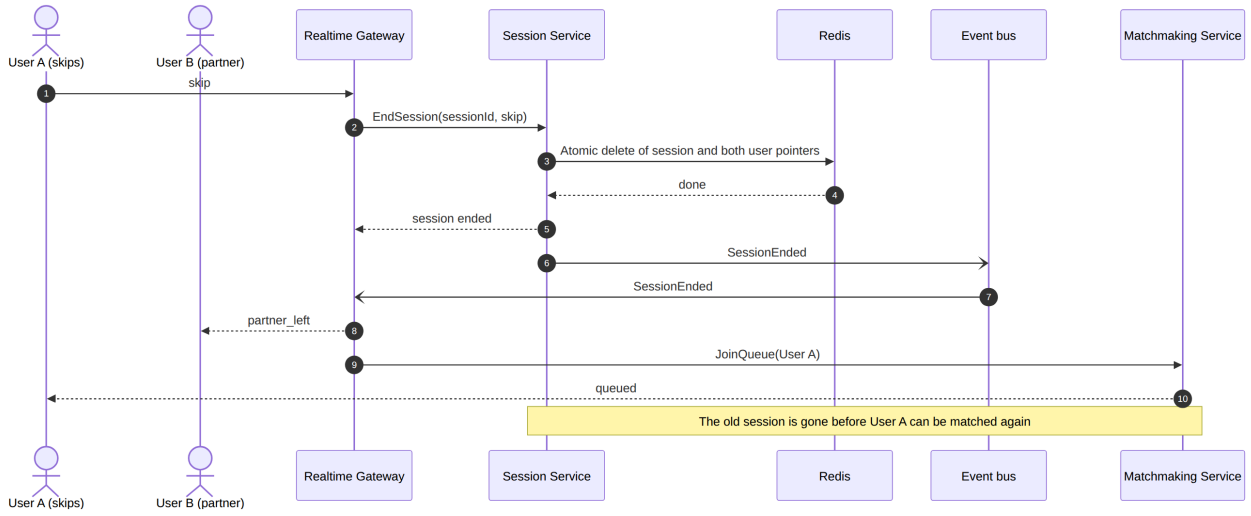


The atomic Redis script is the only place a pairing is decided, so two Matchmaking instances can never assign the same user twice. If no interest match is found within the timeout, the same script falls back to random pairing.

6.5 Session Lifecycle and Skip

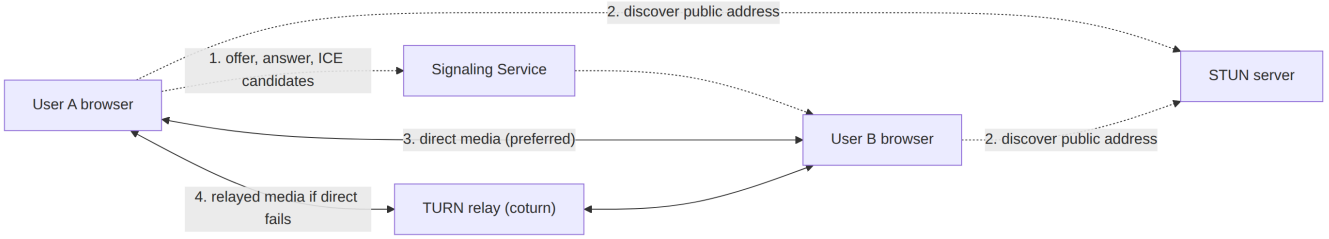


Only the Session Service changes state. Every transition is validated server-side, so signaling for an ended session is rejected.



This ordering is what makes Skip fast and safe: the old match is invalidated first, and late messages for it are rejected.

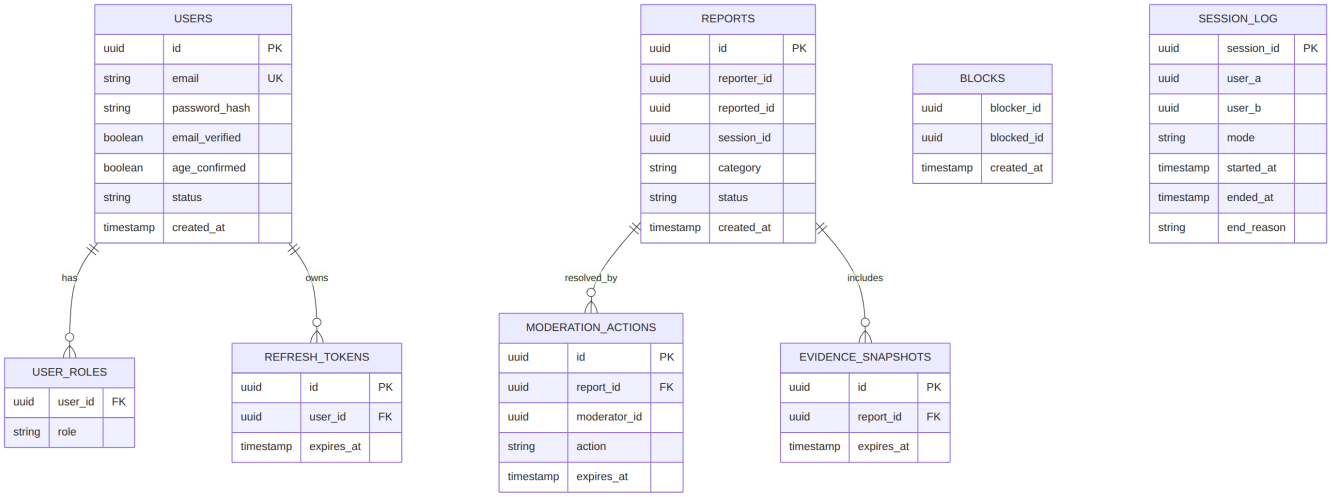
6.6 WebRTC Media Path



Media never passes through the application services. The backend only brokers the connection; if the browsers cannot reach each other directly, regional TURN nodes relay the encrypted media.

6.7 Data Architecture

Each service owns its store. References between services use user IDs only; no foreign key crosses a service boundary.



USERS, USER_ROLES and REFRESH_TOKENS belong to the Identity Service; BLOCKS, REPORTS, EVIDENCE_SNAPSHOTS and MODERATION_ACTIONS to the Safety and Moderation services; SESSION_LOG to the Session Service (metadata only, no media or message content).

Redis key pattern	Owner	Purpose
queue:{mode}:{language}:{bucket}	Matchmaking	Sorted set of users waiting in one shard
user:{id}:state	Matchmaking, Session	Current state (idle, searching, in session) with TTL
session:{id}	Session	Participants, status and timestamps
user:{id}:session	Session	Pointer to the user's current session; enforces one session per user
presence:{id}	Realtime Gateway	Heartbeat key with TTL for stale detection
recent:{id}	Matchmaking	Recently matched partners for cool-down
blocks:{id}	Matchmaking	Cached block list for fast filtering
ratelimit:{scope}:{id}	Gateways, Abuse Control	Counters for rate limiting

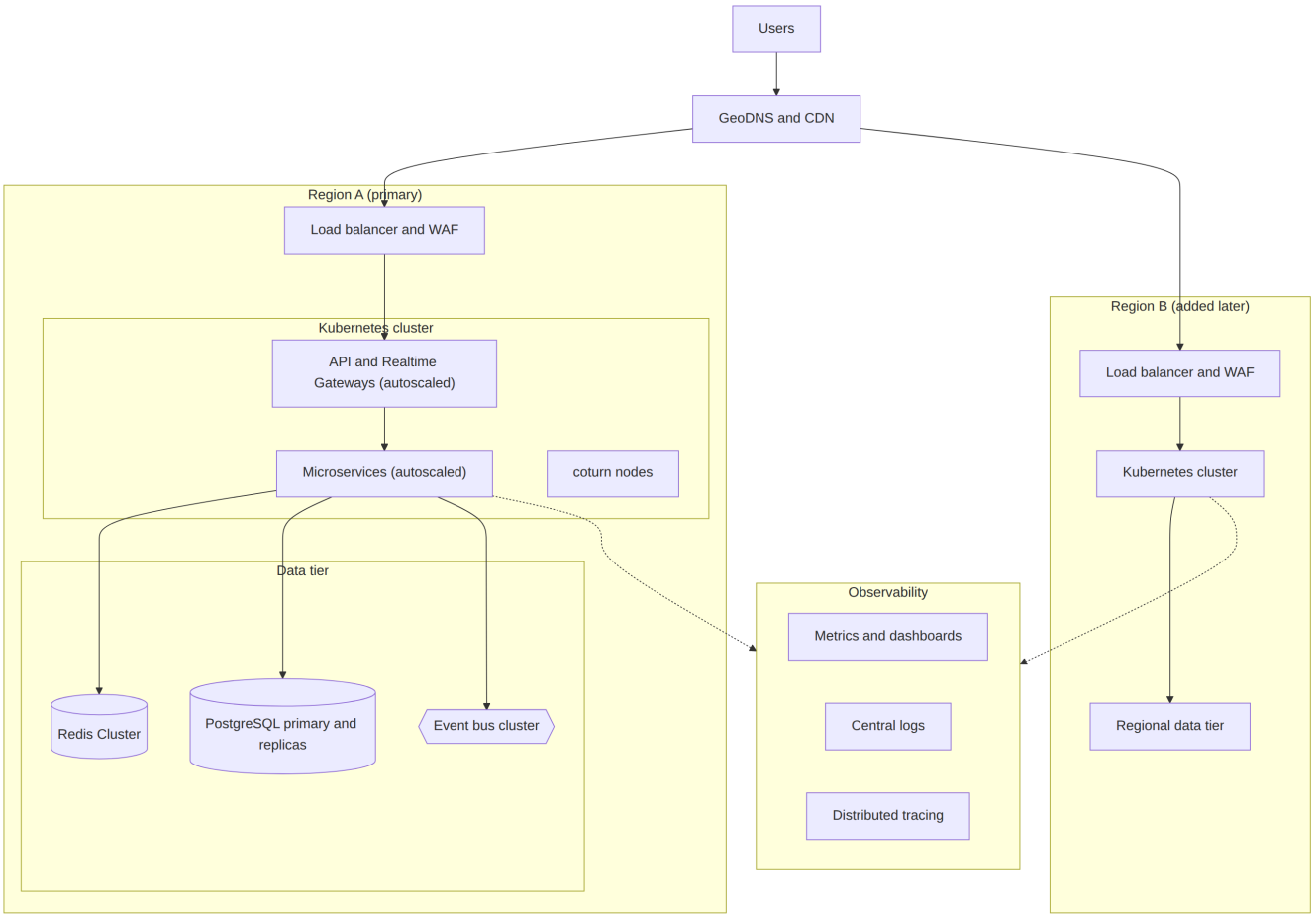
6.8 Event Catalogue

Events are facts published on the bus. Delivery is at-least-once, so every consumer is idempotent.

Event	Published by	Consumed by	Purpose
UserRegistered	Identity	Notification, Analytics	Send the verification email, count signups
UserDeleted	Identity	Safety, Session, Analytics	Remove or anonymise the user's data elsewhere
UserSuspended, UserBanned	Moderation	Identity, Matchmaking, Notification	Block login and matching, notify the user

Event	Published by	Consumed by	Purpose
MatchCreated	Matchmaking	Realtime Gateway, Analytics	Tell both clients to start connecting
SessionStarted	Session	Analytics	Track live sessions and wait times
SessionEnded	Session	Realtime Gateway, Signaling, Safety, Abuse Control, Analytics	Notify the partner, clean caches, keep the last-partner reference for reporting
UserDisconnected, UserReconnected	Realtime Gateway	Session, Presence	Start or cancel the reconnect grace period
ReportFiled	Safety	Moderation, Abuse Control	Queue the report and update risk scores
BlockAdded	Safety	Matchmaking	Update cached block lists
RiskFlagged	Abuse Control	Moderation	Escalate suspicious accounts for review

6.9 Deployment Architecture



Each region is self-contained for matching in the MVP: a user is routed to the nearest region and matched there. Stateless services autoscale on CPU, connections and queue depth, while the data tier scales by adding shards and replicas.

6.10 Resilience and Technology Stack

Pattern	Where it applies
Timeouts and deadlines on every call	All gRPC and HTTP calls between services
Retries with exponential backoff and jitter	Idempotent calls only
Circuit breakers and bulkheads	Calls to Safety, Moderation and Notification, so they cannot slow chat
Outbox pattern	Services that update their database and publish an event in one step
Idempotent consumers and dead-letter queues	Every event subscriber

Pattern	Where it applies
Health and readiness probes, graceful shutdown	All pods; connections drain before a Realtime Gateway stops
Load shedding	Gateways return a busy message instead of accepting work they cannot finish
Degraded modes	Redis outage stops new matching with a clear message; active calls continue because media is peer-to-peer

Layer	Technology
Services	Go, gRPC with Protobuf contracts, REST and WebSocket at the edge
Event bus	NATS JetStream proposed; Kafka is the alternative (see open questions)
Data	Redis Cluster, PostgreSQL per service with migrations, PgBouncer for pooling
Media	Browser WebRTC, coturn for STUN and TURN
Platform	Docker, Kubernetes with autoscaling, service mesh or equivalent for mutual TLS
Observability	OpenTelemetry tracing, Prometheus metrics, Grafana dashboards, structured central logs
Frontend	React, Vue or Svelte with the browser WebRTC API

7. API Overview

7.1 REST Endpoints (indicative)

Method	Path	Service	Purpose
POST	/api/auth/register	Identity	Create account
POST	/api/auth/login	Identity	Obtain tokens
POST	/api/auth/refresh	Identity	Refresh access token
POST	/api/auth/logout	Identity	Invalidate session

Method	Path	Service	Purpose
GET	/api/me	Identity	Current user profile
DELETE	/api/me	Identity	Delete account
GET	/api/turn-credentials	TURN Credential	Short-lived STUN and TURN credentials for a session
GET	/api/stats/online	Presence	Approximate online user count
POST	/api/reports	Safety	Submit a report
POST	/api/blocks	Safety	Block a user
GET	/api/admin/reports	Moderation	Moderator report queue
POST	/api/admin/users/{id}/action	Moderation	Suspend or ban a user

7.2 WebSocket Events (indicative)

Event	Direction	Handled by	Purpose
join_queue (mode, interests, language)	Client to server	Matchmaking	Start searching
leave_queue	Client to server	Matchmaking	Cancel the search
skip	Client to server	Session	End the session and start a new search
offer, answer, ice_candidate	Both	Signaling	WebRTC connection setup
chat_message, typing_start, typing_stop	Both	Signaling	Text chat and typing indicator
heartbeat	Client to server	Realtime Gateway	Keep the connection and session alive
queued, matched (sessionId, role, shared interests)	Server to client	Matchmaking	Search status and match result
partner_left, session_ended	Server to client	Session	Partner skipped or disconnected
error, busy	Server to client	Any service	Failure or overload notice

7.3 Internal Contracts

- **gRPC:** services call each other with Protobuf contracts, a deadline on every call, standard error codes and backward-compatible changes only (add fields, never repurpose them).
- **Events:** every event carries an ID, type, version, timestamp and correlation ID; consumers ignore unknown fields and handle duplicates safely.
- **Contract tests:** each service publishes its contract, and consumers test against it in CI before release.
- **Versioning:** breaking changes ship as a new version running alongside the old one until all consumers have moved.

8. Constraints, Assumptions and Dependencies

8.1 Constraints

- Browser-only delivery for the MVP.
- Go, Redis and PostgreSQL are fixed technology choices; services run as containers on Kubernetes.
- Services communicate over gRPC and an event bus; the bus product (NATS JetStream proposed) is to be confirmed.
- Peer-to-peer media means some users behind restrictive NATs require TURN relay, which adds infrastructure cost.
- A microservice setup needs more tooling (CI/CD, tracing, service discovery) than a single application, so a small platform team is assumed.

8.2 Assumptions

- Users have a working camera, microphone and a reasonably stable internet connection.
- Launch markets and their legal requirements (age assurance, content moderation, data protection) will be confirmed before public release.

8.3 Dependencies

- Email delivery provider for verification and password reset.
- STUN/TURN infrastructure with sufficient bandwidth.
- Hosting with WebSocket and UDP support.
- Moderation staffing or a trust-and-safety partner.

9. Risks and Mitigations

Risk	Impact	Mitigation
Abuse and inappropriate content, including risks to minors	Severe legal and reputational harm	18+ gating, reporting and blocking, moderation team, escalation process, automated screening, age assurance review

Risk	Impact	Mitigation
Distributed complexity: more moving parts and harder debugging	Slower delivery and cascading failures	Tracing with correlation IDs, contract tests, circuit breakers, clear service ownership; ship the ten MVP services first and split further only when metrics justify it
Latency added by service hops	Misses the performance targets	Keep the hot path to three services, use gRPC, co-locate services, enforce latency budgets in load tests
Event bus outage or lost events	Stale moderation or notification state	Clustered persistent streams, dead-letter queues, idempotent consumers; chat never depends on the bus to function
Inconsistent data across services	Wrong ban or block state	Outbox pattern, idempotent handlers, periodic reconciliation jobs
TURN bandwidth costs grow faster than expected	Budget overrun	Monitor relay ratio, autoscale TURN, set per-session bitrate caps
Race conditions in matching	Users in two sessions or ghost sessions	Atomic Redis scripts, single-session pointer, thorough concurrency tests
Stale sessions after network drops	Users stuck, poor experience	Heartbeats, TTLs, grace period and guaranteed cleanup paths
Regulatory change	Rework or market exit	Legal review before launch, configurable compliance features
Low user density at launch causes long waits	Poor retention	Staged launch, queue messaging, interest tags with random fallback
Peer IP exposure	Privacy complaints	User disclosure and optional relay-only mode

10. Testing and Acceptance Criteria

10.1 Test Strategy

- **Unit tests** for matching logic, state machine and rate limiters.
- **Concurrency tests** simulating thousands of simultaneous join and skip actions to prove no user is double-assigned.
- **Contract tests** between every pair of services that call each other or share events.
- **Integration tests** for authentication, signaling relay and session cleanup.
- **Network tests** with simulated packet loss, latency, disconnects and NAT types.
- **Load tests** to 5,000 concurrent users and beyond, with the NFR-PERF targets enforced as pass or fail gates in CI.
- **Chaos tests** that kill service pods, a Redis node and the event bus, and confirm active sessions survive and services recover.
- **Security tests** including authorization checks on signaling and a penetration test before launch.
- **Cross-browser tests** on supported browsers and devices.

10.2 MVP Acceptance Criteria

1. A verified adult user can log in, choose text or video mode, press Start and be connected to another waiting user.
2. Interest tags match users who share one, and fall back to random matching within the timeout.
3. Under concurrent load tests, no user is ever in more than one active session.
4. Pressing Skip ends the current session for both users and begins a new search within 2 seconds.
5. A temporary network drop shorter than the grace period restores the session; a longer drop ends it cleanly for both sides.
6. Blocked users are never matched with each other.
7. Reports reach the moderation queue and a moderator can suspend or ban the reported user, taking effect immediately.
8. No media is recorded or stored by default, and the server does not keep chat message content.
9. At 5,000 concurrent users the load test meets the NFR-PERF targets (matching under 500 ms at the 95th percentile).
10. Stopping any single service instance does not end active calls, and signaling and matching recover automatically.

11. All traffic is encrypted, service-to-service calls use mutual TLS, and rate limits are enforced.

11. Roadmap

Phase	Deliverables
Phase 0: Foundations	Detailed design, Protobuf and event contracts, legal and compliance review, Kubernetes platform, CI/CD, observability stack
Phase 1: MVP	API Gateway, Realtime Gateway, Identity, Matchmaking, Session, Signaling, TURN Credential, Safety, Moderation and Notification services; text and video chat, interest matching, skip, reconnection, reporting and blocking
Phase 2: Hardening	Abuse Control, Presence and Analytics services, load and chaos testing against the SLOs, autoscaling tuning, closed beta
Phase 3: Public launch	Age assurance upgrade, automated moderation, appeals, multi-region readiness, performance tuning
Phase 4: Growth	Language and region filters, college mode, question mode, social login, guest mode if approved, native mobile apps

12. Open Questions

1. Which launch countries, and what age assurance level do they require?
2. Will the platform use a managed TURN service or self-hosted coturn?
3. Who staffs moderation, and what are the target response times for reports?
4. Event bus: NATS JetStream or Kafka?
5. Managed Kubernetes or self-operated, and on which cloud?
6. Should the optional guest mode (anonymous, text-only) ever be offered, given the mandatory-authentication choice?
7. What are the retention periods for reports and evidence snapshots?
8. When should a second region be added?

13. Recommendation Summary

Build the classic Omegle-style experience first, as a small set of focused microservices: start with the ten MVP services, keep the live chat path to three of them, and split further only when load or team size demands it. Use Go, Redis, PostgreSQL and an event bus to make matching race-free, fast and scalable. Treat safety, moderation, privacy and performance targets as core features from day one rather than add-ons.